



MAINTENANT PEUT TU DÉFINIR POINT PAR POINT les étapes d'un cahier des charges avec une présentation générale du projet la stack technique pour une version non déployée, PostgreSQL, Docker un conteneur par partie back/front/BDD, BDD PostgreSQL sous conteneur, back sous Django/Python, front en type script et react + Tailwind CSS

de plus définir point par point les missions et détailler les

Voici la trame complète de votre Cahier des Charges Technique, structurée point par point pour votre stage. Elle intègre la stack technique demandée et détaille chaque mission pour vous guider pas à pas dans le développement.

CAHIER DES CHARGES TECHNIQUE – LUXCLEAN SERVICES

1. Présentation Générale du Projet

Le projet consiste à développer une plateforme web sur-mesure pour "Luxclean Services", une entreprise de nettoyage. Le site a deux objectifs principaux :

- **Front-Office (Site Public) :** Agir comme un outil de génération de leads optimisé pour la conversion (demandes de devis unifiées B2B/B2C, candidatures de recrutement, blog/portfolio des interventions).
- **Back-Office (CRM Interne) :** Fournir un tableau de bord sécurisé aux équipes pour traiter les demandes avec un système de traçabilité de lecture, analyser l'affluence, et gérer les permissions (Super Admin vs Admin classique).

Note : Ce projet est développé dans un cadre de stage et sera livré dans un environnement local (non déployé en production pour le moment).

2. Architecture et Stack Technique (Environnement Local)

L'application sera divisée en micro-services locaux grâce à Docker Compose, garantissant un environnement de développement isolé et reproductible.

- **Conteneur 1 : Base de Données**
 - Technologie : PostgreSQL (Image officielle Docker).
 - Rôle : Stockage persistant et sécurisé des données relationnelles.
- **Conteneur 2 : Backend (API REST)**
 - Technologies : Python, Django, Django REST Framework (DRF).
 - Rôle : Gestion de la logique métier, requêtes ORM vers PostgreSQL, système d'authentification (JWT), et exposition de l'API.
- **Conteneur 3 : Frontend (SPA - Single Page Application)**
 - Technologies : React, TypeScript, Tailwind CSS (via Vite.js).
 - Rôle : Interfaces utilisateurs publiques et interface d'administration sécurisée.

3. Détail des Missions (Plan de Développement)

Mission 1 : Initialisation de l'environnement (DevOps local)

Objectif : Mettre en place l'architecture Docker.

- **Tâche 1.1 :** Créer le fichier `docker-compose.yml` définissant les 3 services (db, backend, frontend).
- **Tâche 1.2 :** Configurer les variables d'environnement (fichiers `.env`) pour la connexion entre Django et PostgreSQL.
- **Tâche 1.3 :** Mettre en place les volumes Docker pour garantir la persistance des données PostgreSQL même si le conteneur est redémarré.
- **Tâche 1.4 :** Créer les `Dockerfile` respectifs pour le backend (installation des paquets Python) et le frontend (installation de Node/NPM).

Mission 2 : Modélisation et Backend BDD (Django)

Objectif : Traduire le schéma conceptuel en modèles Django ORM.

- **Tâche 2.1 :** Configurer le modèle `User` personnalisé dans Django (Custom User Model) pour inclure le rôle (Super Admin vs Admin).
- **Tâche 2.2 :** Créer les modèles ORM pour `Quote` (Devis), `JobApplication` (Candidatures), `Post` (Blog), `Setting` (Paramètres) et `Analytics`.
- **Tâche 2.3 :** Générer et exécuter les migrations Django vers PostgreSQL (`makemigrations / migrate`).

- **Tâche 2.4 :** Créer un script de "Seed" (fixtures) pour peupler la base avec un compte Super Admin par défaut ("Loyde") et des fausses données pour tester.

Mission 3 : Développement de l'API REST (Django DRF)

Objectif : Créer les routes de communication entre le front et la base de données.

- **Tâche 3.1 (Auth) :** Mettre en place l'authentification par token (JWT - JSON Web Tokens) pour sécuriser l'accès au Back-Office.
- **Tâche 3.2 (Routes Publiques) :** Créer les endpoints `POST /api/quotes/` et `POST /api/jobs/` pour recevoir les formulaires du site vitrine, et `GET /api/posts/` pour afficher le blog.
- **Tâche 3.3 (Routes Admin - CRM) :** Créer les endpoints protégés pour lister les devis/candidatures.
- **Tâche 3.4 (Logique métier complexe) :** Créer une route `PATCH /api/quotes/{id}/read` qui passe un message en lu et enregistre automatiquement l'ID de l'admin authentifié qui a fait la requête.
- **Tâche 3.5 (Permissions) :** Configurer les classes de permissions DRF pour que seul le Super Admin puisse requêter les routes `GET/POST/DELETE /api/users/`.

Mission 4 : Développement Frontend - Site Vitrine (React / TS)

Objectif : Coder l'interface publique orientée conversion.

- **Tâche 4.1 :** Initialiser le projet avec Vite, configurer TypeScript, et installer Tailwind CSS.
- **Tâche 4.2 :** Créer les composants UI réutilisables avec Tailwind (Boutons, Champs de formulaire, Cartes, Navbar).
- **Tâche 4.3 :** Développer la page d'Accueil et la page des Services.
- **Tâche 4.4 :** Développer le Formulaire de Devis unifié (B2B/B2C) avec validation des champs (ex: React Hook Form + Zod) et envoi via `fetch` ou `axios` vers l'API Django.
- **Tâche 4.5 :** Développer l'Espace Recrutement (formulaire + upload de CV) et la page Blog (fetch des articles depuis l'API).

Mission 5 : Développement Frontend - Tableau de Bord CRM (React / TS)

Objectif : Créer l'espace d'administration sécurisé.

- **Tâche 5.1 :** Créer le système de routing protégé (Private Routes) : l'accès nécessite un JWT valide.
- **Tâche 5.2 :** Développer la page "Boîte de réception" (Devis et Candidatures). Implémenter le changement visuel (gras/normal ou pastille couleur) selon si le statut est lu ou non, et afficher "Lu par : [Nom]" (visible uniquement si Super Admin).
- **Tâche 5.3 :** Développer la vue Analytics (Dashboard simple affichant le nombre de visites et de devis).
- **Tâche 5.4 :** Développer la page de gestion des paramètres (mise à jour du mail/WhatsApp).

- **Tâche 5.5 :** Développer la gestion des Administrateurs (accessible uniquement pour Loyde) avec ajout/suppression de comptes "Admin".

Mission 6 : Finalisation et Documentation

Objectif : Rendre un projet propre et soutenable.

- **Tâche 6.1 :** Nettoyage du code, vérification du typage strict TypeScript.
- **Tâche 6.2 :** Écriture d'un fichier `README.md` détaillé (commandes Docker pour lancer le projet, identifiants du Super Admin par défaut).
- **Tâche 6.3 :** Tests fonctionnels locaux (remplissage d'un devis côté front, connexion admin, vérification du passage en "lu").

Ce document constitue une excellente "feuille de route" pour votre stage. Il démontre à vos évaluateurs que vous maîtrisez l'ensemble du cycle de développement (Fullstack + DevOps de base).